Il livello MicroArchitetturale

A cura di Casola Valentina

Obiettivo del livello microarchitetturale

- Obiettivo del livello microarchitetturale è l'implementazione del livello ISA (Instruction Set Architecture);
- La realizzazione "scelta" (ne esistono differenti) dipende da alcune caratteristiche fondamentali:
  - Costo,
  - Performance,
- Alcune macchine RISC hanno istruzioni talmente semplici che possono essere eseguite in un *unico* ciclo di clock.

Come progettare il livello architetturale di una macchina virtuale (1)

- Approccio *metodologico*;
- Approccio *operativo*;
- In questa parte del corso seguiamo un *approccio operativo;*
- Introdurremo i principi alla base della progettazione usando un esempio di architettura reale (un sotto insieme delle istruzione della Java Virtual Machine – IJVM)

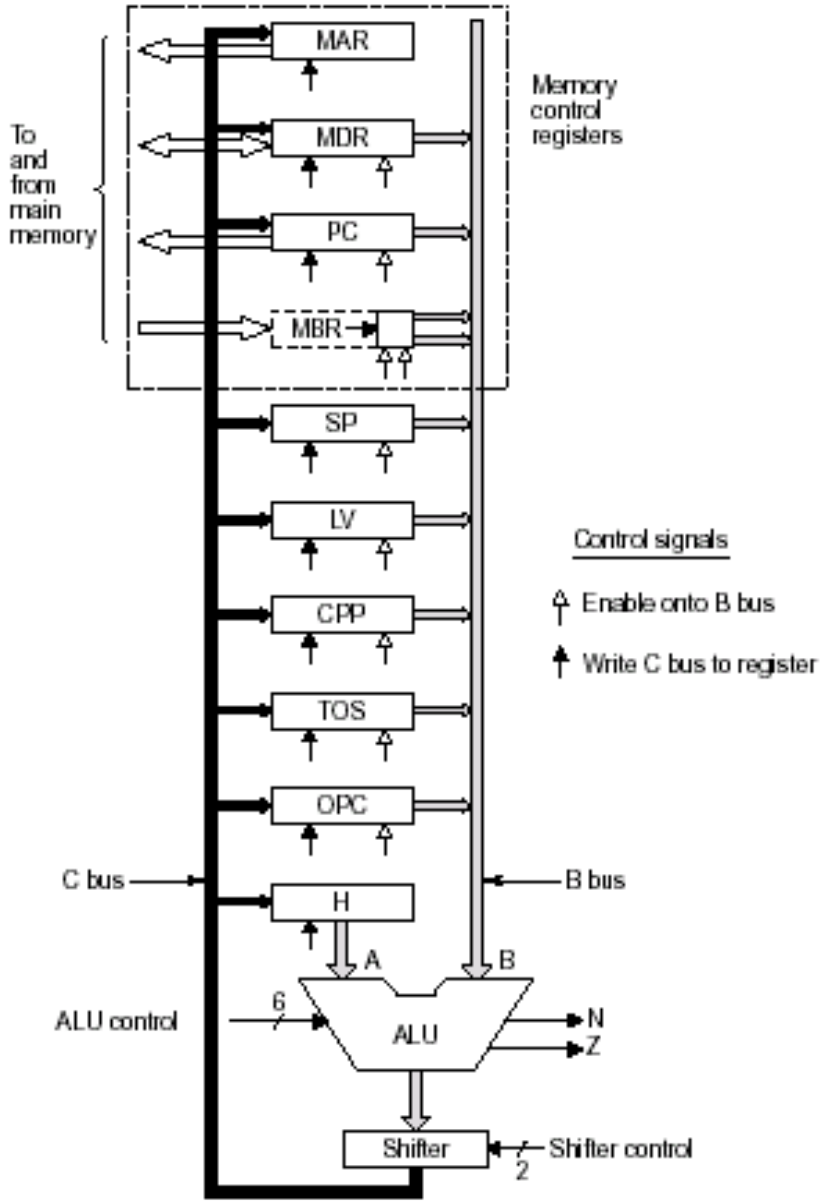
Come progettare il livello architetturale di una macchina virtuale (2)

- Una microarchitettura deve contenere un microprogramma in grado di prelevare decodificare ed eseguire una istruzione IJVM;
- Un microprogramma è costituito dalla sequenza delle funzioni e procedure che deve essere invocata da un programma principale per poter compiere una istruzione (fetch-decode-execute).
- Il microprogramma deve essere in grado di pilotare nella giusta sequenza i “circuiti logici” che costituiscono la macchina fisica reale.
- Il microprogramma è contenuta in una memoria ROM.

Come progettare il livello architetturale di una macchina virtuale (3)

- Il microprogramma ha un insieme di variabili che determinano lo **stato** del computer,
- esse sono accessibili a tutte le microistruzioni e possono essere modificate da queste ultime.
- Il formato delle istruzioni è un aspetto fondamentale per la microarchitettura; la IJVM ha istruzioni *corte* con *pochi campi (OPCODE, OPERAND)*

Il data path della microarchitettura MIC

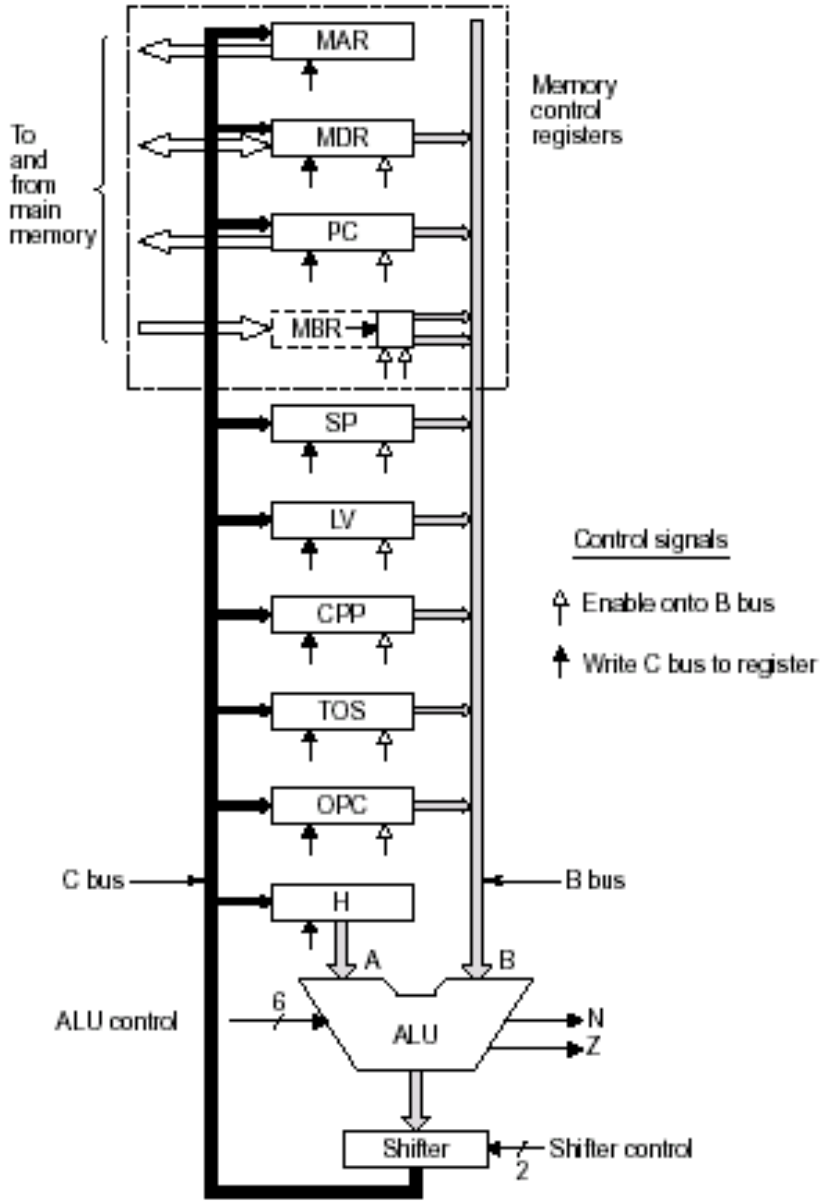


Il datpath è ottimizzato per il particolare set di istruzioni ISA;

Ogni microistruzione controlla il datapath per un ciclo;

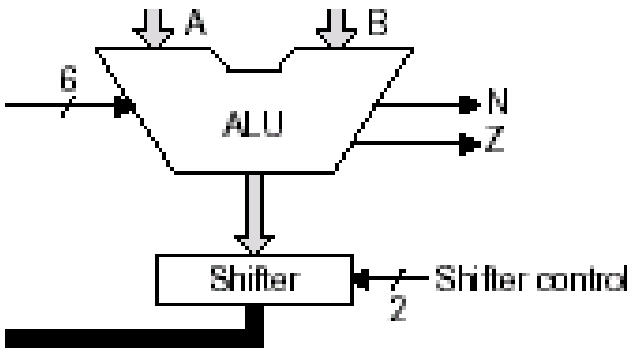
La lista di microistruzioni forma le microprocedure del microprogramma.

Caratteristiche del Datapath del MIC1



- Registri a 32 bits accessibili solo a livello microarchitetturale,
- 2 BUS (uno di lettura dai registri - B, l'altro per la scrittura sui registri - C)
- Q: come faccio a scrivere da un registro in H?

La ALU



- La ALU ha 6 linee di controllo:
- F0, F1 per l'operazione;
- ENA, ENAB per abilitare gli inputs A e B;
- INVA per invertire l'ingresso A (left - registro H);
- INC per incrementare di 1;
- Teoriche 64 combinazioni (non tutte utilizzate).
- Lo shift register ha altre due linee di controllo: Shift left logical (SLL8 - riempie il byte ms con tutti zero, Shift Right arithmetic (SRA1 - sposta di un bit e lascia invariato il bit più significativo)

F_0	F_1	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\overline{A}$
1	0	1	1	0	0	$\overline{B}$
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B – A
1	1	0	1	1	1	B – 1
1	1	1	0	1	1	–A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	–1

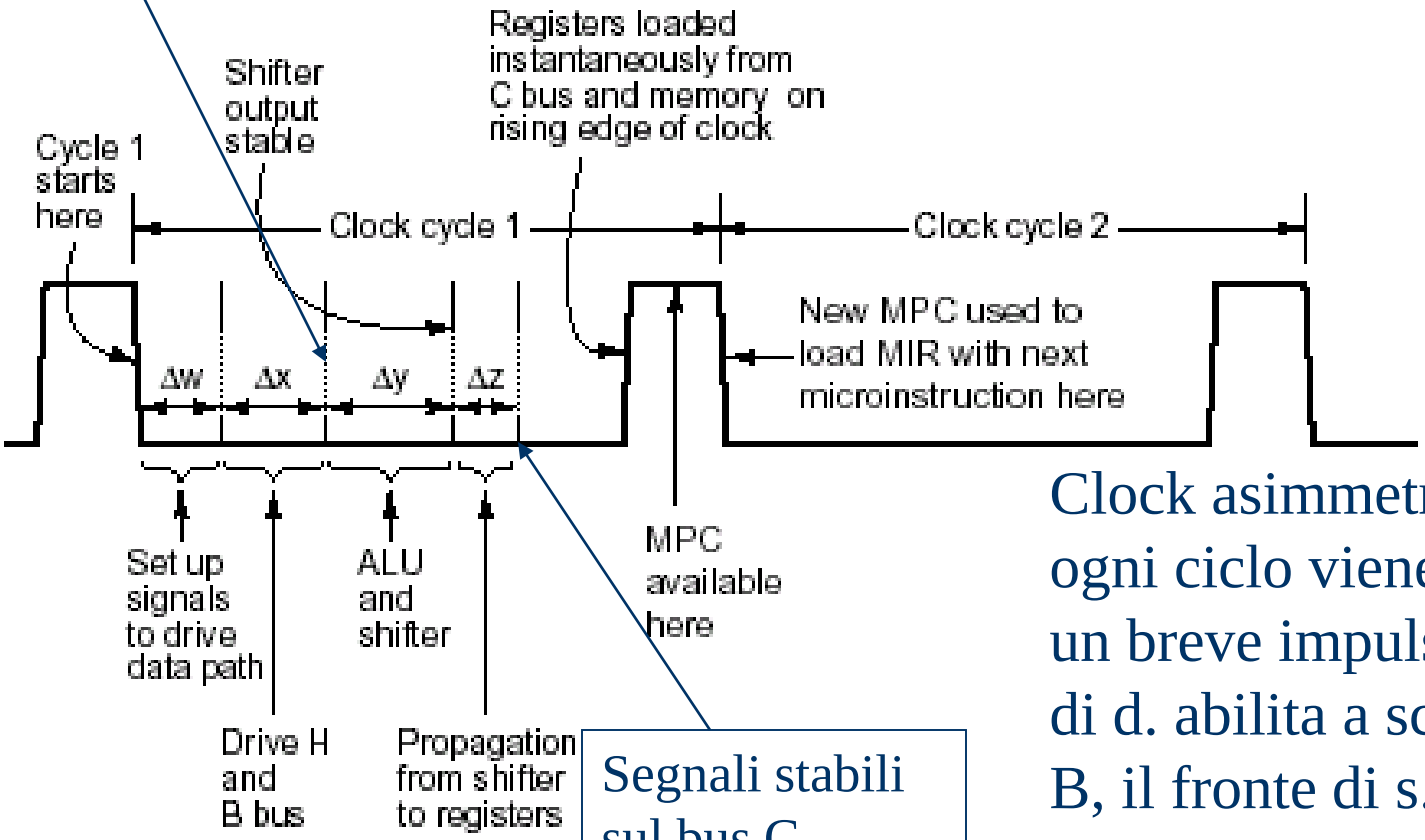
Fa solo passare il contenuto di B

E' possibile leggere e scrivere da e nello stesso registro in un unico ciclo di path

Alcune combinazioni utili dei 6 segnali della ALU e l'operazione risultatnte

Timing diagram di un ciclo del data path

Segnali stabili sul bus B



Segnali stabili sul bus C

Clock asimmetrico, ad ogni ciclo viene generato un breve impulso; il fronte di d. abilita a scrivere su B, il fronte di s. abilita a scrivere su un registro

Generazione di un clock asimmetrico

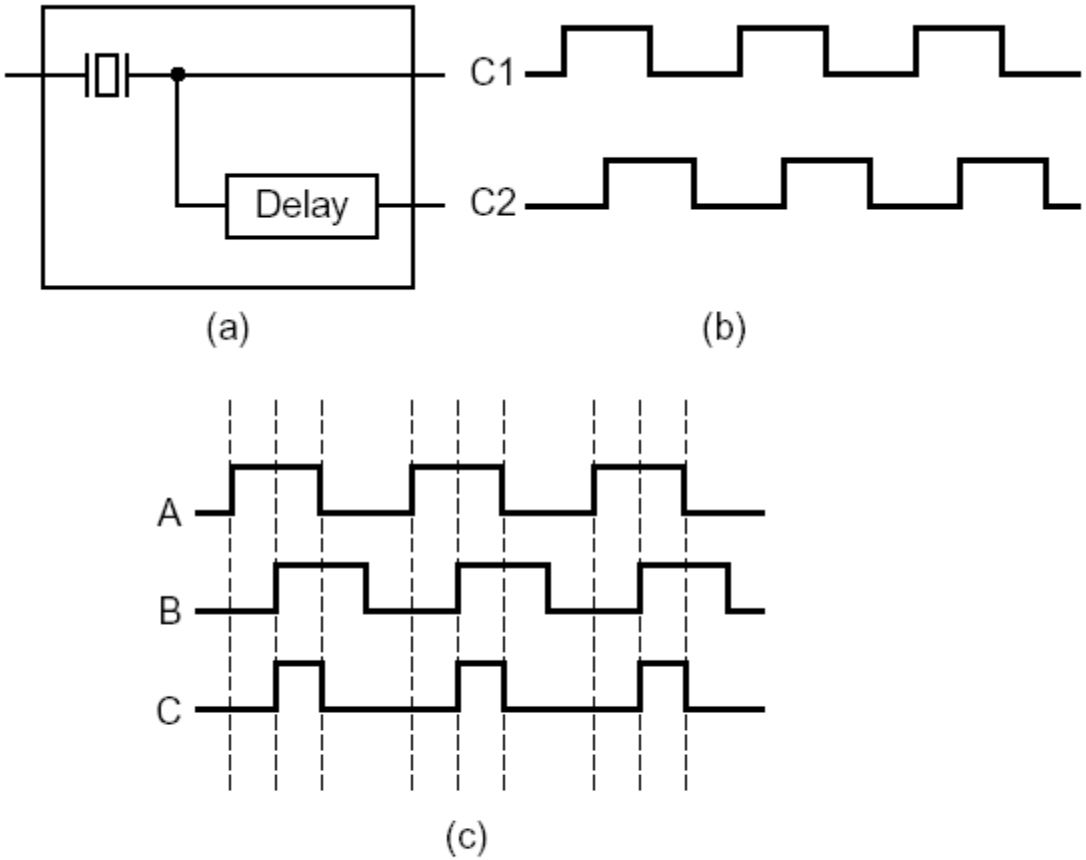
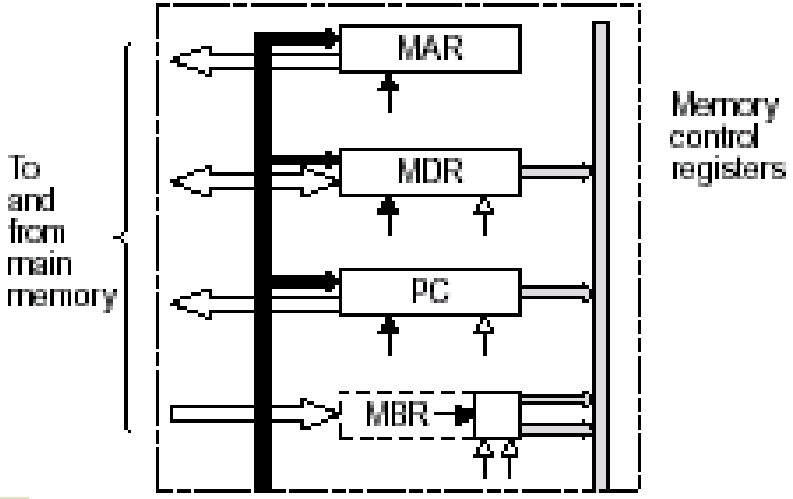


Figure 3-21. (a) A clock. (b) The timing diagram for the clock. (c) Generation of an asymmetric clock.

Le *microfasi* del clock

- Il clock deve essere opportunamente dimensionato affinchè $\Delta w + \Delta x + \Delta y + \Delta z$ sia minore di T (periodo del Clock).
- I vari Δi sono tutti ritardi di propagazione o tempi combinatori (sufficientemente stimabili), gli unici istanti "abilitanti" sono i due fronti (salita e discesa) del clock.
- È compito del progettista dimensionare opportunamente T in funzione dei vari Δi della microarchitettura.

Comunicazione con la memoria



Il MIC 1 ha due modi diversi per comunicare con la memoria:

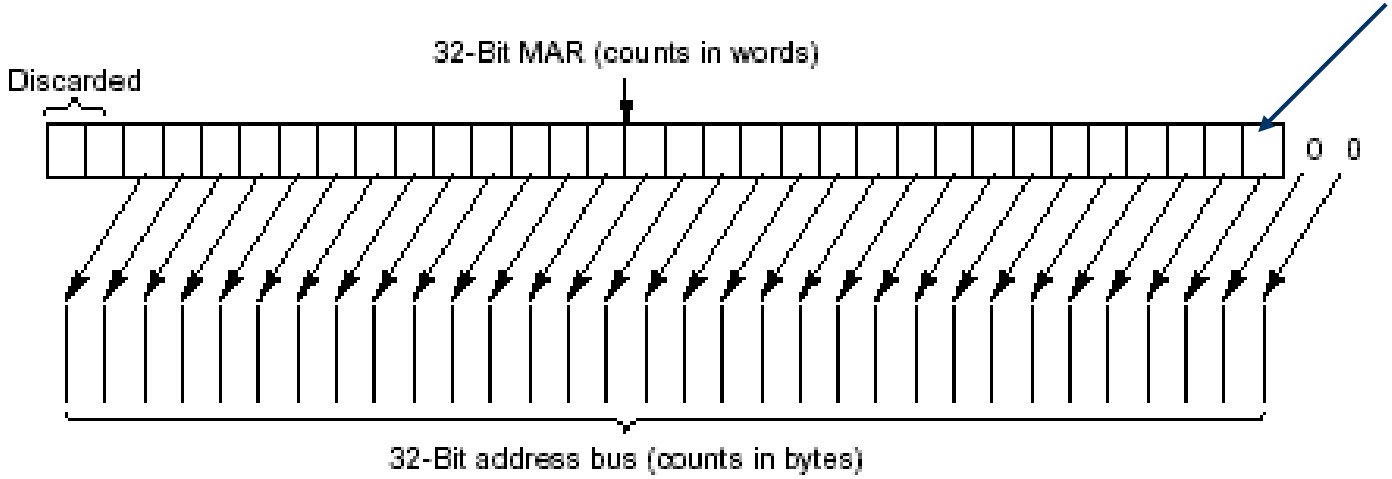
1 porto a 32 bit (indirizzabile a word) controllato da MAR/MDR;

1 porto ad 8 bit (indirizzabile a byte) controllato da PC/MBR - monodirezionali;

MAR e PC indirizzano due parti differenti della memoria

Mapping dei bits del MAR sulle linee degli indirizzi del bus

Indirizzi multipli di 4



- La memoria è organizzata a byte, per accedere ad una word di 32 bit si utilizza un trucco;
- MBR è un byte, prima di leggerlo dal bus B devo "estenderlo a 32 bit"; posso farlo in due modi (con o senza estensione del segno) con i due segnali di controllo MBR, MBRU.

Le microistruzioni per il Mic-1 (1)

- Per controllare il dataPath del Mic 1 occorrono 29 segnali raggruppati in 5 tipologie:
  - 9 segnali per scrivere dal bus C ai registri;
  - 9 segnali per scrivere dal registri al bus B;
  - 8 segnali per la ALUe lo Shift register;
  - 2 segnali per indicare il read/write per MAR/MDR;
  - 1 segnale per indicare il memory fetch PC/MBR;
- Occorrono 29 segnali;

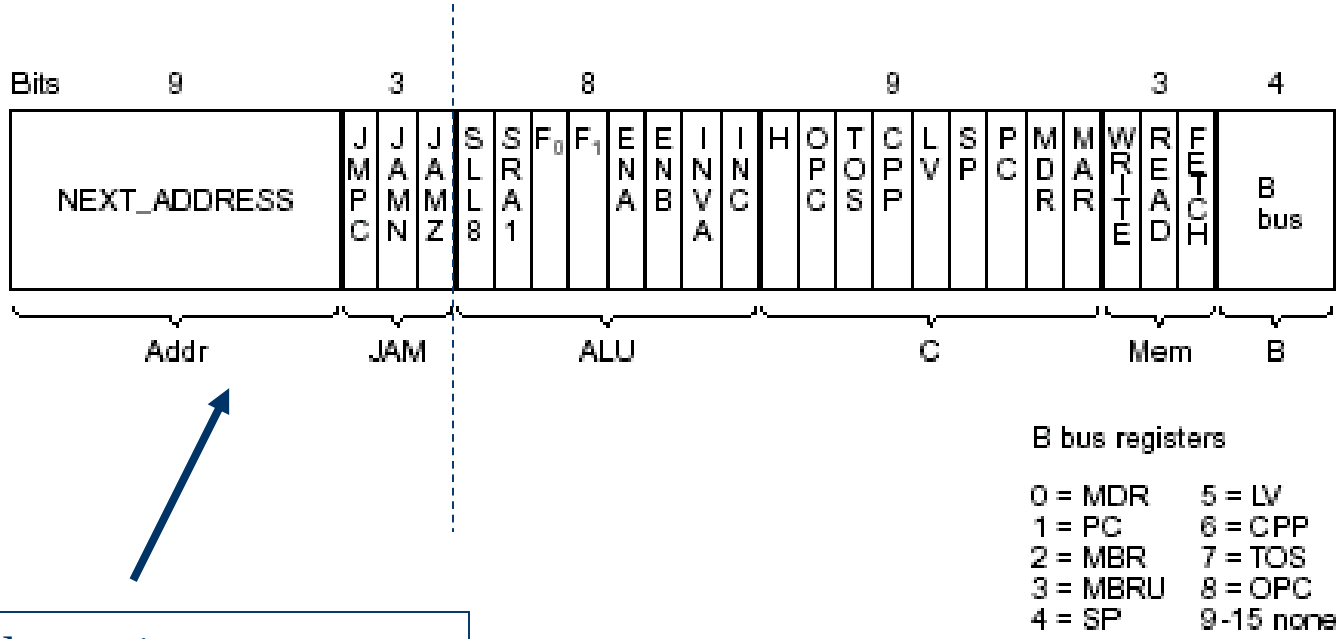
Le microistruzioni per il Mic-1 (2)

- Una operazione di Read dalla memoria può iniziare solo alla fine del ciclo, cioè solo dopo che il registro MAR è stato caricato;
- Dopo una operazione di Read, i dati saranno disponibili in un registro interno solo alla fine del ciclo successivo (cache hit=100%);
- Se attivo un segnale di Read al ciclo *k*, i dati possono essere usati solo al ciclo *k+2*;
- Al ciclo k+1 posso usare il dataPath per operazioni che non coinvolgono la lettura dalla memoria.
- Posso leggere contemporaneamente la memoria con PC e con MAR ma se viene indirizzata la stessa parte di memoria ci possono essere delle situazioni indefinite.

Le microistruzioni per il Mic-1 (3)

- Posso ridurre i bit della control word (CW) perché ci sono dei segnali mutuamente esclusivi (esempio lettura da un registro su B) -> per 9 segnali posso usare solo 4 bit ed usare un decoder per generare 16 segnali di controllo (di cui 7 inutilizzati);
- L'ordine dei bit nella CW viene scelto per ottimizzare l'attraversamento dei cavi sulla piastra.

Formato delle microistruzioni per il Mic-1



B bus registers

0 = MDR	5 = LV
1 = PC	6 = CPP
2 = MBR	7 = TOS
3 = MBRU	8 = OPC
4 = SP	9-15 none

Come determinare l'indirizzo della microistruzione successiva (Addr + JAM)

Il controllo delle microistruzioni

- Ogni CW controlla il datapath in ogni ciclo di clock; per determinare la sequenza delle microistruzioni necessarie per eseguire una istruzione ISA, occorre una unità di controllo con un *sequencer* che produce due tipi di info:
  - Lo stato di ogni segnale di controllo nel sistema,
  - L'indirizzo della microistruzione successiva da eseguire.

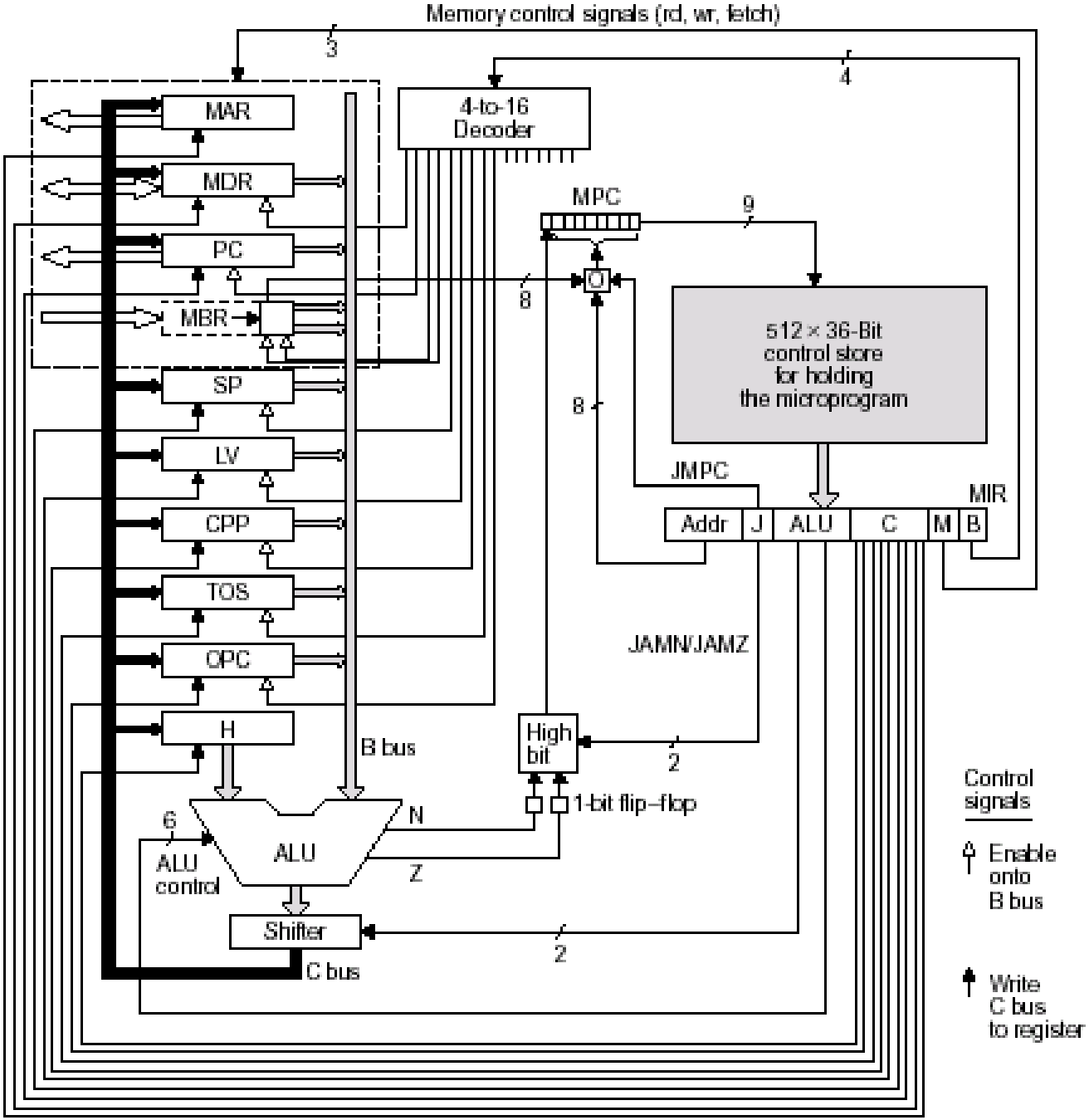


Diagramma a blocchi completo per la Mic-1

L’unità di controllo (1)

- La parte principale è costituita da una *control store* di 512 word da 36 bits.
- Le microistruzioni non sono memorizzate in ordine, serve un meccanismo più flessibile (dato che le sequenze delle microprocedure sono brevi, si può implementare un meccanismo più efficiente): ogni microistruzione esplicita il suo successore.

L’unità di controllo (2)

- Per accedere alla control store si usano due registri MPC (anche se non serve un contatore) e MIR;
- Il MIR viene caricato sul fronte di discesa del clock, poi le fasi del clock procedono come visto nel diagramma delle tempificazioni.
- Quando la ALU ha completato la sua elaborazione, lo **shift register** ed **N** e **Z** sono stabili; N e Z vengono salvati in una coppia di 1-bit flip flops sul fronte di salita del clock.
- N e Z vengono usati per calcolare la microistruzione successiva

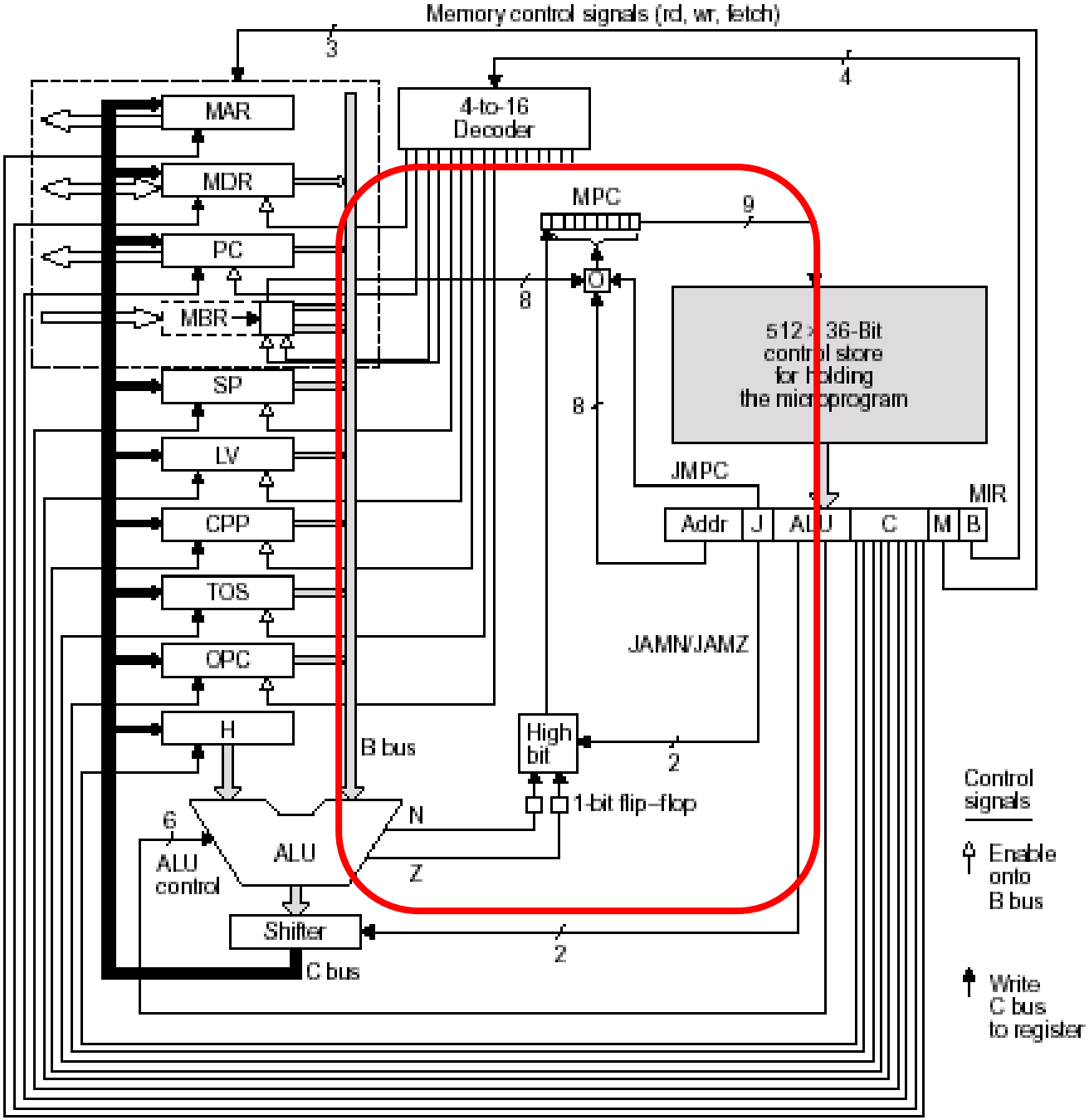


Diagramma a blocchi completo per la Mic-1, come calcolare MPC

Come determinare l'istruzione successiva (1)

- Il calcolo dell'istruzione successiva comincia appena il MIR è stabile:
  1. i 9 bit del campo NEXT_ADDRESS vengono copiati in MPC e contemporaneamente si valuta il campo JAM;
     1. Se JAM = "000" non si deve fare nulla ed MPC="NEXT_ADDRESS";
     2. Se almeno 1 bit di JAM è diverso da "0", occorre calcolare l'MPC.

Come determinare l'istruzione successiva: effetto di JAMZ e JAMN (1)

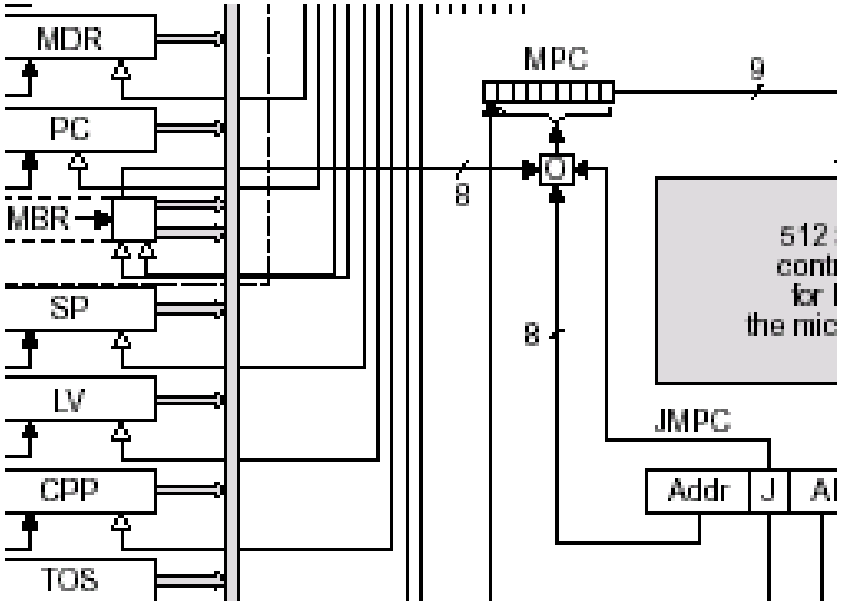
- Se JAMN è settato ad "1" -> il bit più significativo di MPC viene messo in OR con il flag N;
- Se JAMZ è settato ad "1" -> il bit più significativo di MPC viene messo in OR con il flag Z;
- Se entrambi sono settati ad "1" -> il bit più significativo di MPC viene messo in OR con entrambi
- F = (JAMN AND N) OR (JAMZ AND Z) OR bitNEXT_ADDRESS[8]

Come determinare l'istruzione successiva: effetto di JAMZ e JAMN (2)

- In ogni caso, NEXT_ADDRESS rimane invariato oppure il suo bit più significativo [8] viene messo in OR con "1".
- Ovviamente se bitNEXT_ADDRESS[8] = 1, l'OR non ha nessun effetto su MPC.
- Esempio :
  - NEXT_ADDRESS = [0 1 0 0 1 0 0 1 0] 0x092,
  - se JAMZ e/o JAMN = 1 →
  - MPC = [1 1 0 0 1 0 0 1 0] 0x192.

Come determinare l'istruzione successiva: effetto di JAMC

- Se JAMC è settato ad "1" -> gli 8 bit di MBR sono messi in OR bit a bit con gli 8 bits meno significativi di MIR;



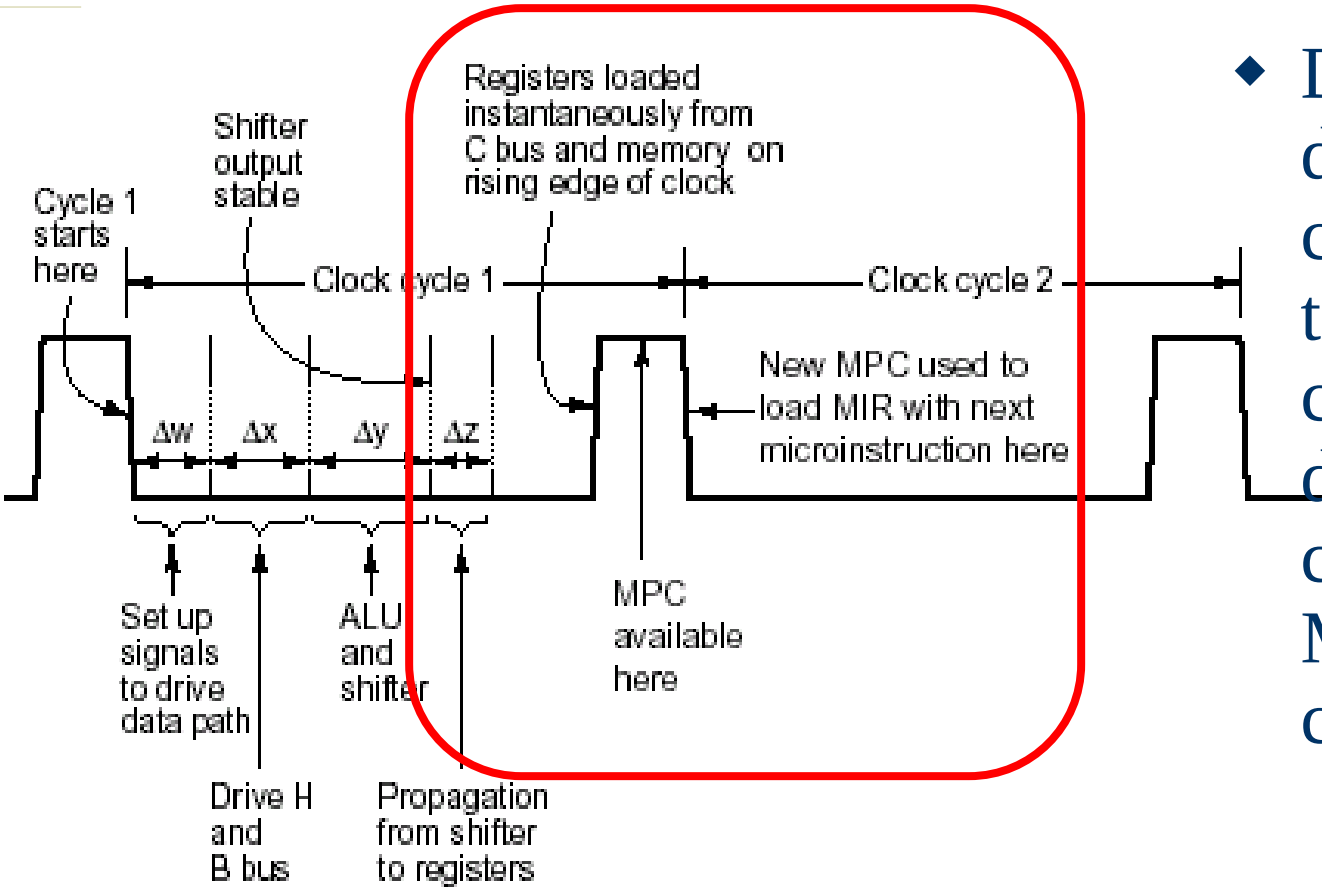
Come determinare l'istruzione successiva: effetto di JAMC (2)

- Mettere in OR MBR e NEXT_ADDRESS, permette una efficiente gestione dei salti multiway,
- MBR contiene un Opcode, non un microindirizzo……

Osservazione importante

- Nella IJVM, l'indirizzo nella Control store della prima microistruzione di una istruzione ISA corrisponde al valore binario del codice della istruzione stessa:
- Esempio se ADD è codificata con 00000001 (0x01) allora la sua prima microistruzione è stata memorizzata all'indirizzo 0x01.

Caricamento della prossima microistruzione



- Dopo il fronte di salita del clock deve trascorrere un certo intervallo di tempo prima che il nuovo MPC venga caricato

Riferimenti bibliografici

- A.S. Tanenbaum…..
  - CAP 4: The microarchitecture level